Computación

Clase 7

Temas

- Programación estructurada
- Subprogramas
- Funciones

Programación estructurada

La Programación estructurada es un paradigma de programación donde se enfatiza el orden y agrupamiento de operaciones a realizar.

Programación estructurada

- El paradigma estructurado tiene, entre otras, las siguientes características :

a) El programa completo tiene un diseño modular.

b) Los módulos se diseñan con metodología descendente, también denominada “top-down”.

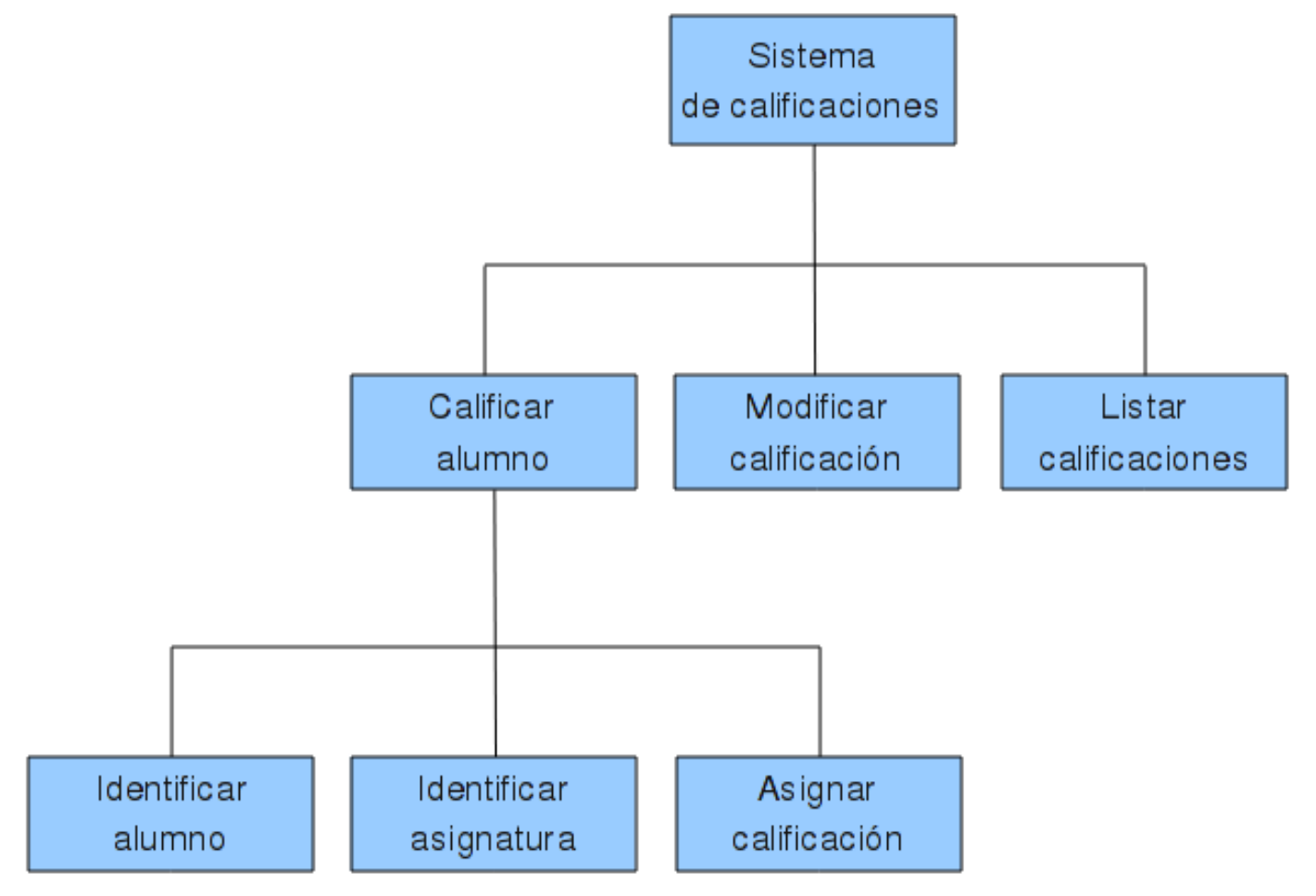
c) Cada módulo se codifica utilizando las tres estructuras de básicas de control (secuenciales, selectivas y repetitivas)

Diseño modular

- División de un programa en partes más manejables (usualmente denominadas subprogramas o módulos)

- Ventajas:
  - ✓ Menor costo
  - ✓ Trabajo simultáneo en paralelo
  - ✓ Reutilización

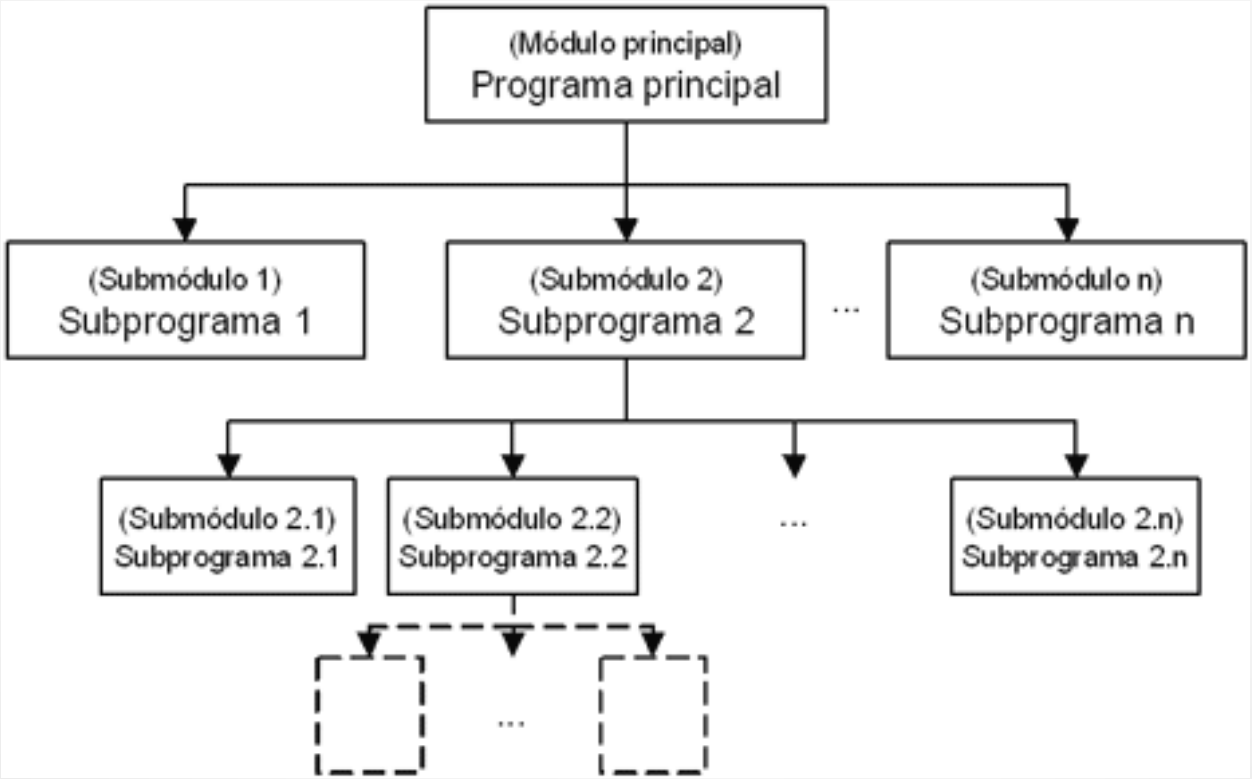
Programación estructurada



Subprogramas

Un subprograma es una sección del programa (idealmente autónoma), que realiza una tarea definida, la cual puede ser llamada (activada, invocada) por otras secciones del programa, siempre que se necesite, a fin de ejecutar la tarea asignada.

Subprogramas



Subprogramas

- Ventajas
  - Ahorro de código
  - Modularidad
  - Buen mantenimiento

Tipos de subprogramas

Ejemplos de subprogramas predefinidos

clrscr; {procedimiento que borra la pantalla}

writeln(A); {procedimiento que permite escribir}

B:=**Sqr**(C*D); {función que calcula el cuadrado de un valor}

If **Ord**(‘B’) > 50 then {función que obtiene el valor ASCII de un caracter}

writeln(**Sqrt**(x):6:2); {función que obtiene la raíz cuadrada de un valor}

Funciones

Una función es un subprograma que recibe como parámetros los datos que requiere para cumplir su objetivo y devuelve un resultado al invocante (programa ppal o subprograma) que es el valor de esa función.
Una función siempre retorna un ***único*** *valor.*

Declaración de una función

PROGRAM identificador
USES
  identificador/es
CONST
  Definición de constantes
TYPE
  Declaración de tipos
VAR
  Declaración de variables

{Definición de funciones}
FUNCTION Identificador……..

BEGIN

  Sentencias ejecutables
END.

Declaración de una función

Encabezamiento

FUNCTION identifi_funcion (P_1, ...P_n):Tipo_de_dato ;
{Declaraciones locales}
Begin
.....

.....
.....
.....
identifi_funcion := valor_de_la_función;
end;

Tipo de dato del valor que devuelve la función

Lista de parámetros formales

Asignación al identificador de la función del valor de retorno

Parámetros formales

Son variables de algún tipo determinado cuyo objetivo es especificar la comunicación del subprograma invocante con la función invocada.

Ejemplo

```
function maximo(a,b : real):real;

 begin
  If a > b Then
      maximo := a
  else
      maximo := b
 end;
```

Invocación o llamado a una función

```
Program XX;
   Var
     a:real;
     b1,b2:integer;
   Function ejemplo(x,y:integer):real ;
   Begin
     .........
     ejemplo:= .........;
    End;
   Begin
    .......
    a:=ejemplo(b1,b2)+12.04;
    ........

   End.
```

1

2

La invocación (en una expresión) que activa la función; ésta es ejecutada, luego el valor guardado en ejemplo es devuelto al p.p.

Hallar la suma de los enteros entre 1 y un valor límite.

```
Program EjFuncion;
  Var
      n1:integer;

Function suma(limite:integer):integer;
   Var
        num, acum:integer;
   Begin
        acum:=0;
        For num:=1  to limite do
           acum:=acum +num;
        suma:=acum
   End;

Begin
         Writeln('Ingrese límite');
         Readln(n1);
         Writeln(suma(n1));
End.
```

DESARROLLO

Asignación VALOR

INVOCACION

Dado un número entero positivo, determinar si el mismo es primo

```
Program primo;
var
  num:integer;
function EsPrimo(numero: integer): boolean;
var
  i,tope: integer;
begin
  i:= 2;
  tope:= trunc(sqrt(numero));
  while (i<=tope) and not (numero mod i  = 0) do
      i:= i+1;
  EsPrimo:= i > tope;
end;
Begin
 readln(num);
 if EsPrimo(num) then
   writeln('el nro ',num,' es primo')
 else
   writeln('el nro ',num,' no es primo')
End.
```